



Universitatea
Transilvania
din Braşov

FACULTATEA DE INGINERIE ELECTRICĂ
ŞI ŞTIINŢA CALCULATOARELOR

Purcariu Bogdan – Product Owner & Business Analyst

Horvath Daiana – Frontend Developer

Popescu Călin – Backend Developer

Gavrilescu Raul – AI Developer

Banciu Vicentiu-Alexandru – QA Tester

MediStock Application

Cuprins

1	Introduction.....	4
1.1	Definitions.....	4
1.2	Objectives.....	4
2	Product definition.....	6
2.1	Product vision board	6
2.2	Stakeholders	7
2.3	User Personas and User Stories.....	8
2.4	Release plan	9
3	Business Analysis	10
3.1	Market Analysis	10
3.2	Competitor Comparison.....	11
3.3	SWOT Analysis	11
3.4	Feasibility study	13
4	system requirements.....	14
4.1	Functional requirements.....	14
4.1.1	FR-01 — Authentication and Authorization.....	15
4.1.2	FR-02 — Role-Based Access Control.....	15
4.1.3	FR-03 — Centralized Medication Visibility.....	15
4.1.4	FR-04 — Transfer-Based Inventory Management.....	15
4.1.5	FR-05 — Transfer Request Creation.....	15
4.1.6	FR-06 — Dual Confirmation Process	15
4.1.7	FR-07 — Automatic Inventory Synchronization	15
4.1.8	FR-08 — Expiration and Lot Tracking	15
4.1.9	FR-09 — Dashboard and Alert System.....	15
4.1.10	FR-10 — AI-Assisted Forecasting.....	15
4.1.11	FR-11 — Transfer History and Logging.....	15
4.2	Non-Functional requirements.....	16
4.2.1	Scalability.....	16
4.2.2	Usability	16
4.2.3	Security.....	16
4.2.4	Performance.....	16
4.3	Business Rules	17
4.3.1	BR-01 — Expired medications cannot be actively used or transferred.	17
4.3.2	BR-02 — Medication inventory updates must occur through transfer operations.17	
4.3.3	BR-03 — Transfer requests require approval from both parties.....	17
4.3.4	BR-04 — Stock quantities cannot become negative.....	17

4.3.5	BR-05 — Medication transfers cannot exceed available inventory.....	17
4.3.6	BR-06 — Users can only perform actions permitted by their assigned role.....	18
4.3.7	BR-07 — Inventory modifications must be traceable.....	18
4.3.8	BR-08 — AI recommendations act as decision-support mechanisms only.	18
4.3.9	BR-09 — Medication lot information must remain associated with inventory records.....	18
4.3.10	BR-10 — Inventory information must remain synchronized after transfer completion.....	18
5	System Design & Architecture	18
5.1	High-level Architecture.....	19
5.2	Stateless Architecture.....	20
5.3	User workflow – Medication	21
6	Backend Development.....	22
6.1	API Contract.....	22
6.2	Entity Design.....	23
6.3	Authentication.....	24
6.4	Business Logic	24
7	Frontend.....	25
7.1	UI Design	25
7.2	Components.....	26
7.3	Frontend–Backend Communication.....	26
8	AI Development.....	27
8.1	AI Objective.....	27
8.1.1	Monte Carlo Confidence Estimation.....	30
8.1.2	AI Outputs	30
8.1.3	AI Execution Workflow	30
8.2	Dataset.....	31
8.3	Explainability.....	31
8.4	AI Testing.....	32
9	Testing and validation.....	32
9.1	Functional Testing.....	32
9.2	API Testing	33
9.3	Integration Testing.....	33
9.4	Test Scenarios	34
9.5	Validation Results	34
10	Future Improvements.....	35
11	Conclusions.....	36

1 INTRODUCTION

1.1 Definitions

MediStock represents a medication inventory management platform developed to support healthcare institutions, hospitals and pharmacy environments in organizing, monitoring and optimizing medication-related activities. The project aims to provide a centralized and integrated environment capable of managing the complete lifecycle of medications, starting from inventory registration and stock updates to expiration tracking, provider interaction and intelligent demand prediction. The system is designed to simplify operational workflows while improving visibility over medication availability and reducing inefficiencies associated with manual inventory management processes. Through a modular architecture and a modern software stack, the platform seeks to provide an accessible alternative to large-scale enterprise healthcare ecosystems while maintaining sufficient flexibility for future expansion and feature integration.

1.2 OBJECTIVES

The primary objective of MediStock is to improve operational efficiency within healthcare environments by addressing common issues associated with medication management. Hospitals and pharmacy departments frequently encounter difficulties such as inaccurate stock visibility, medication shortages, expiration-related waste, inconsistent inventory updates and the lack of predictive mechanisms capable of supporting procurement decisions. Traditional inventory approaches often rely heavily on manual processes and fragmented workflows, which increase the risk of human error and reduce responsiveness to changing operational needs. MediStock attempts to address these limitations through a software-centric solution focused on inventory transparency and intelligent planning mechanisms.

From a functional perspective, the platform introduces a medication lifecycle management approach centered around structured inventory handling. Medications are treated as managed entities whose stock levels, associated lots and expiration information can be continuously monitored and updated. Instead of relying solely on static records, the system aims to capture operational changes occurring throughout medication usage, reception and distribution processes. Special emphasis is placed on lot-level tracking, since medications originating from different suppliers or delivery periods may possess different expiration dates, quantities and batch characteristics. This design decision allows the platform to more accurately reflect real-world hospital inventory operations and creates the foundation required for more advanced functionalities such as expiration monitoring and predictive analysis.

The platform also incorporates role-based access control mechanisms intended to support organizational responsibilities within healthcare environments. Different users interact with the system according to specific permissions aligned with their operational responsibilities. Hospital managers are responsible for overall system supervision, user management and high-level

inventory administration. Pharmacists participate in day-to-day inventory activities such as medication updates, stock management and lot handling, while providers operate as external entities responsible for supply interactions and medication delivery workflows. Additional read-only roles may be introduced for users whose activities are limited to inventory observation and reporting purposes. This separation of responsibilities contributes to both operational clarity and improved system security.

Security and architectural scalability represent important aspects of the platform design. MedStock adopts a stateless architecture using token-based authentication mechanisms through JWT technologies and REST-based communication between frontend and backend services. Unlike traditional session-oriented implementations, the system avoids storing user session information directly on the server, allowing requests to remain independent and improving scalability and fault tolerance. This architectural decision reduces memory overhead while facilitating easier maintenance and deployment, particularly in environments where infrastructure resources may be limited. The separation between presentation, business and persistence layers also contributes to improved maintainability and future extensibility.

Another important component of the platform consists of dashboard and reporting functionalities designed to improve operational awareness. Healthcare personnel frequently require immediate visibility regarding critical inventory situations, including low stock quantities or medications approaching expiration. To address these needs, MedStock integrates monitoring and alert mechanisms capable of highlighting operational risks and providing users with visual indicators regarding inventory conditions. Reports generated by the platform may provide usage statistics, inventory summaries and analytical insights intended to support operational decisions and improve resource planning.

A significant differentiating aspect of the project involves the integration of an AI-assisted forecasting component intended to support proactive inventory planning. The forecasting module aims to analyze historical inventory behavior and identify trends associated with medication consumption across hospital departments. Rather than relying exclusively on static thresholds, the system attempts to estimate future medication demand and recommend inventory adjustments capable of reducing shortages or overstock situations. The forecasting functionality is intended as a decision-support mechanism rather than a replacement for human judgment and may evolve toward explainable recommendation systems capable of providing users with contextual reasoning behind prediction outcomes.

From a strategic perspective, MedStock positions itself differently from large-scale enterprise healthcare ecosystems such as Oracle Health and Omnicell. Rather than attempting to provide a complete hospital management environment involving electronic health records, large-scale clinical workflows and extensive hardware automation, the project intentionally focuses on medication inventory management and operational simplicity. Large enterprise platforms frequently require significant infrastructure investments, specialized hardware integrations and complex deployment procedures. MedStock instead follows a lightweight and modular approach intended to simplify adoption and reduce implementation complexity, particularly for smaller healthcare environments, pilot deployments or academic scenarios.

The long-term vision of the project includes future expansion opportunities involving mobile support, provider workflow integration, barcode-based medication identification, cloud deployment and broader healthcare ecosystem interoperability. Additional AI capabilities, predictive analytics modules and advanced reporting mechanisms may further enhance the platform over time. By maintaining a modular architecture and clearly separated components, the system aims to support gradual evolution while preserving maintainability and reducing implementation complexity. Overall, MedStock represents an attempt to combine modern software engineering principles, healthcare workflow requirements and intelligent inventory assistance into a unified and adaptable platform.

2 PRODUCT DEFINITION

2.1 PRODUCT VISION BOARD

The Product Vision Board was developed to define the strategic direction, target users, core problems, proposed solutions and long-term objectives of the MediStock platform. Its purpose was to establish a common understanding among team members regarding the product scope and expected value before beginning technical implementation activities. The vision board served as an initial alignment tool between business objectives and technical development decisions, allowing stakeholders to identify both operational needs and future opportunities for system expansion.

The vision of MediStock is to assist healthcare personnel in managing medication inventories through a centralized and intelligent platform capable of monitoring stock availability, medication usage and expiration-related information. The system aims to reduce medication waste and improve operational efficiency by providing better visibility over inventory conditions and supporting informed decision-making processes.

The primary users identified during the product definition stage include pharmacists, hospital managers and providers. Pharmacists represent the operational users responsible for inventory updates and medication handling activities. Hospital managers require broader inventory visibility and reporting capabilities in order to supervise organizational resources and identify operational risks. Providers act as external stakeholders involved in supply-related workflows and medication delivery processes.

Several key problems were identified during the early analysis phase. One of the primary challenges involves medications reaching expiration before being utilized, resulting in financial losses and operational inefficiencies. Additional issues include medication overconsumption, insufficient inventory visibility and the risk of stock shortages that may affect healthcare services and patient treatment continuity.

To address these problems, MediStock proposes a centralized inventory solution focused on medication stock tracking, expiration monitoring and proactive alert mechanisms. The platform aims to provide a unified environment where users can monitor medication status, identify inventory risks and receive notifications regarding critical operational situations.

The core feature set currently defined for the system includes medication management functionalities implemented through CRUD operations, dashboard and alert components designed for operational visibility and an AI forecasting module intended to support predictive stock

planning activities. These functionalities establish the foundation of the platform and represent the initial product scope considered for the first development releases.

From a business perspective, MediStock seeks to reduce medication waste and minimize the occurrence of stock shortages. By improving inventory visibility and supporting more efficient stock planning processes, the system attempts to reduce operational costs and improve resource allocation efficiency within healthcare environments.

Several success metrics were also defined in order to evaluate project impact and validate system effectiveness. These metrics include reducing medication waste, increasing stock availability and reducing operational expenses associated with inventory inefficiencies and medication losses.

Certain operational constraints were identified during the planning phase. The system assumes daily usage within healthcare environments and depends significantly on users entering inventory information consistently and accurately. Since forecasting and monitoring functionalities rely heavily on available data quality, incorrect or inconsistent input may directly affect system performance and prediction reliability.

Finally, future development directions were considered as part of the long-term product vision. Potential extensions include the development of a mobile application and the introduction of a business-to-consumer component capable of expanding the platform beyond internal hospital workflows. These improvements reflect the intention of maintaining a scalable and adaptable architecture capable of supporting future organizational and technological requirements.



Fig. 1. MediStock Production Vision Board

2.2 STAKEHOLDERS

The MediStock platform involves several categories of stakeholders, each having a specific role in the medication inventory management process. The main stakeholders identified for this project are the Hospital Manager, the Pharmacist, the Provider and the Viewer. These roles were defined in order to clearly separate responsibilities, access permissions and operational activities inside the system.

The Hospital Manager represents the user with the highest level of control within the platform. This stakeholder is responsible for supervising the system, managing users and ensuring that the medication inventory process is correctly configured and maintained. The Hospital Manager can create, update, delete and view medication records, manage user accounts and assign roles according to organizational responsibilities. This role is important because it provides administrative control over the platform and ensures that only authorized users can perform sensitive operations.

The Pharmacist represents the main operational user of the system. This stakeholder is directly involved in daily medication management activities, such as adding medications, updating stock information, managing lots and monitoring expiration dates. The Pharmacist has permissions similar to the Hospital Manager in terms of medication and stock management, but does not have access to user management functionalities. This separation allows pharmacists to perform their operational duties while maintaining administrative control at the hospital management level.

The Provider represents an external stakeholder involved in the supply process. This role is associated with medication delivery and stock replenishment workflows. In the current scope of the platform, the Provider mainly has read-only access to relevant medication or supply-related information and may be involved in future workflows such as viewing reorder requests, confirming availability or supporting delivery confirmation processes. The Provider does not directly modify the hospital's internal stock, in order to preserve data integrity and operational control.

The Viewer represents a read-only role designed for users who need visibility over medication information without being allowed to modify system data. This stakeholder may include hospital managers, supervisors or other authorized personnel who need access to dashboards, reports or medication availability information. The Viewer role supports transparency and decision-making while reducing the risk of unauthorized changes to medication or stock records.

Overall, the stakeholder structure supports a role-based access model that reflects real operational responsibilities within a healthcare environment. By separating administrative, operational, external and read-only responsibilities, MedStock ensures better control over sensitive actions, improves accountability and provides a clearer workflow for medication inventory management.

2.3 USER PERSONAS AND USER STORIES

To better understand the needs, expectations and operational responsibilities of future users, several user personas were defined during the product planning phase. These personas were created to represent realistic stakeholder profiles and to support the identification of functional requirements, workflow decisions and system interactions. By defining representative users and their objectives, the development team was able to better align system functionality with real healthcare scenarios and improve user-centered design decisions.

One of the primary personas identified for the platform is **Maria, a Hospital Pharmacist**, who represents the main operational user of MediStock. Maria is responsible for handling medication inventory activities on a daily basis and requires immediate visibility regarding stock levels, medication availability and expiration status. Her responsibilities include adding medications, updating quantities, managing medication lots and monitoring products approaching expiration. Since her work directly influences inventory accuracy and medication availability, she requires an interface that is intuitive, fast and capable of presenting critical information clearly.

Another important persona is **Alex, a Hospital Manager**, whose responsibilities extend beyond operational inventory activities. Alex supervises the overall system, manages users and

permissions and monitors organizational performance through reports and dashboards. Unlike pharmacists, managers require broader access to system configuration and management functionalities. Their primary objective is to ensure that operational workflows function correctly while maintaining proper access control and overall inventory supervision. The system also considers **David, a Medication Provider**, representing an external stakeholder responsible for medication supply and delivery interactions. Providers are interested in inventory requests, supply visibility and future communication workflows associated with medication replenishment activities. Although providers have fewer permissions than internal hospital users, they remain an important part of the broader medication ecosystem. Based on these personas, a collection of user stories was defined to capture expected user interactions and system behavior. User stories were used to describe functional requirements from the perspective of end users and to provide a clearer understanding of business needs. For example, a pharmacist may require the ability to monitor expiration dates in order to prevent medication waste and ensure patient safety. This requirement can be represented through the user story: *As a pharmacist, I want to receive notifications regarding medications approaching expiration so that I can take preventive actions before products become unusable.* Similarly, managers require inventory visibility and user management capabilities. An example user story associated with this role is: *As a hospital manager, I want to manage users and permissions so that I can maintain secure and organized access control throughout the platform.*

Inventory planning and forecasting requirements also generated user stories related to the AI component of the platform. An example of such functionality may be expressed as: *As a pharmacist, I want the system to predict future medication demand so that inventory shortages can be reduced and procurement decisions can be improved.*

By combining user personas with user stories, the project team was able to establish a user-centered development process focused on operational needs and real-world healthcare scenarios. These artifacts provided an important foundation for backlog creation, requirement prioritization and feature planning throughout the development lifecycle.

2.4 RELEASE PLAN

The MediStock development process follows an incremental release strategy designed to gradually expand the platform functionality while maintaining a stable and manageable implementation process. Instead of attempting to deliver all functionalities simultaneously, the project was divided into multiple releases, each introducing a new level of system maturity and operational capability. This approach allows the development team to validate core functionalities early, gather feedback and progressively extend the platform according to both technical priorities and business objectives.

The first release focuses on delivering the core functionality required to establish a functional prototype and validate the main workflow of the system. At this stage, the platform concentrates on medication inventory management and operational interactions between healthcare entities. One of the primary objectives of the initial release is to support medication transfer processes between hospitals and pharmacies, allowing users to manage and monitor inventory movement across organizational units. In addition, the system includes medication creation and administration functionalities, enabling users to add, update and manage medication-related information within a centralized environment. The first release also introduces the initial AI implementation responsible for supporting forecasting and intelligent inventory analysis capabilities. This component represents an early attempt to integrate predictive functionality into medication management workflows and establish the foundation for future analytical features.

The second release is intended to improve operational visibility and workflow automation capabilities. At this stage, the system is expected to introduce more advanced dashboard functionalities, reporting components and alert mechanisms designed to provide users with immediate feedback regarding inventory status and operational risks. Features such as low-stock notifications, expiration monitoring improvements and enhanced visualization capabilities may be introduced in order to support decision-making processes and improve inventory awareness throughout healthcare environments.

The third release represents the long-term strategic direction of the platform and focuses on scalability and ecosystem expansion. Beyond improving AI forecasting capabilities and introducing more advanced analytical mechanisms, this release aims to extend the platform beyond traditional hospital workflows. One of the major future objectives involves expanding the application toward a business-to-consumer model and introducing patient-oriented functionalities into the ecosystem. Rather than limiting the platform to managers, pharmacists and providers, future iterations may include patients as active participants capable of interacting with medication information and healthcare-related services. This transition would allow the platform to evolve from an internal inventory management solution into a broader healthcare ecosystem supporting multiple categories of users and use cases.

By following a staged release strategy, MedStock aims to balance implementation complexity with long-term scalability. The incremental approach enables the development team to focus on delivering immediate value while preserving flexibility for future expansion and adaptation to evolving healthcare requirements.

3 BUSINESS ANALYSIS

3.1 MARKET ANALYSIS

The healthcare sector increasingly relies on digital technologies in order to improve operational efficiency, reduce costs and optimize resource allocation processes. Hospitals and pharmacy environments face several recurring challenges associated with medication inventory management. Among the most common issues are medication shortages, medication waste caused by expiration, inconsistent inventory visibility and difficulties associated with maintaining accurate stock information. Traditional approaches frequently rely on manual inventory processes and fragmented workflows that increase the probability of human errors and reduce operational responsiveness.

To address these challenges, several enterprise healthcare platforms have emerged and currently dominate the market. Among the most relevant competitors identified during the market analysis phase are **Omnicell** and **Oracle Health**. These platforms provide large-scale healthcare solutions capable of supporting medication inventory activities and broader healthcare processes. However, their primary focus is generally oriented toward large organizations and enterprise environments with significant infrastructure capabilities.

MedStock was designed as an alternative approach focused specifically on medication inventory workflows. Rather than attempting to replicate complete healthcare ecosystems, the platform concentrates on a narrower and more specialized problem space associated with medication tracking, inventory visibility and intelligent stock planning. The project aims to provide a lightweight and modular solution that remains easier to deploy and maintain while preserving the ability to support future scalability.

3.2 COMPETITOR COMPARISON

During the **competitor analysis process**, Omnicell and Oracle Health were selected due to their strong presence in the healthcare technology market and their similarities with certain aspects of the proposed system.

Omnicell provides advanced medication inventory management capabilities together with automated dispensing systems, expiration monitoring and predictive inventory analysis. One of the major strengths of Omnicell lies in its extensive hardware ecosystem, which includes automated dispensing cabinets, inventory automation technologies and robotic systems capable of supporting medication distribution workflows. These technologies improve operational efficiency and reduce human intervention during medication handling processes. However, the platform introduces additional infrastructure complexity due to its dependency on specialized hardware systems. Such requirements may increase implementation costs and reduce accessibility for smaller hospitals or organizations with limited infrastructure resources.

Oracle Health approaches healthcare management from a broader perspective and provides a complete hospital ecosystem rather than focusing exclusively on medication inventory activities. The platform supports patient administration, healthcare workflows, operational analytics and large-scale organizational management. Oracle Health also integrates enterprise-level analytics and AI-assisted functionalities designed to optimize healthcare operations. Although the platform offers extensive capabilities, its architecture is oriented toward extremely large-scale deployments and introduces significant operational complexity. The implementation and maintenance process may become difficult for smaller healthcare environments that require simpler and more focused solutions.

Unlike these competitors, MedStock does not attempt to replace complete healthcare ecosystems or large-scale automation platforms. Instead, the project focuses specifically on medication inventory processes and simplified operational workflows. The platform introduces AI-assisted forecasting mechanisms specialized for medication demand analysis and expiration-aware inventory planning. The architecture follows a lightweight and modular approach intended to reduce deployment complexity and improve accessibility for smaller hospitals or pilot healthcare environments.

While enterprise platforms such as Omnicell and Oracle Health provide broad functionality including hardware integration and complete healthcare ecosystems, MedStock differentiates itself through simplicity, modular design, medication-centered workflows and lower operational complexity. Additionally, the platform was designed to minimize infrastructure requirements and maintenance efforts, making it more suitable for organizations operating with limited technological resources.

3.3 SWOT ANALYSIS

A **SWOT analysis** was conducted in order to identify the internal strengths and weaknesses of the MediStock platform while also evaluating external opportunities and threats that may influence its development and adoption. This analysis supports strategic planning activities and provides a broader understanding of the project's competitive positioning within healthcare environments.

One of the primary strengths of MediStock lies in its role-based access control model, which enables the separation of responsibilities among different categories of users. By assigning permissions according to organizational roles such as Hospital Manager, Pharmacist and Provider, the platform improves operational security and creates a more structured workflow. Another

important strength is the focused scope of the project. Unlike enterprise healthcare ecosystems that attempt to solve numerous healthcare problems simultaneously, MediStock concentrates specifically on medication inventory workflows, allowing the development team to address a well-defined operational problem. The platform also benefits from AI-assisted forecasting capabilities that support proactive inventory planning and contribute toward reducing stock shortages and medication waste. Additionally, the modular architecture of the system allows future expansion and easier maintenance, making the project more adaptable to future technological requirements.

Despite these advantages, several weaknesses were identified during the analysis process. Since the forecasting component depends on historical information, limited data availability may affect the accuracy and reliability of AI predictions during the early stages of deployment. Another limitation is the absence of hardware automation technologies commonly used in larger healthcare ecosystems. Unlike enterprise competitors that integrate robotics, RFID systems or automated dispensing cabinets, MediStock currently relies on software-only processes. The platform also depends heavily on manual user input, meaning that inaccurate or inconsistent inventory updates may negatively affect operational workflows and analytical results. Finally, uncertainty regarding AI prediction accuracy remains a challenge, particularly because forecasting models require sufficient and reliable training data to produce meaningful recommendations.

Several opportunities were identified that may positively influence the future growth of the platform. The healthcare sector increasingly adopts digital technologies as organizations seek more efficient and intelligent operational processes. This growing demand for healthcare digitization creates favorable conditions for inventory-focused systems such as MediStock. Additionally, the platform may later evolve toward multi-hospital deployments, allowing broader organizational adoption and larger operational ecosystems. Future improvements involving advanced reporting systems and analytical dashboards may further increase the value delivered to users. Furthermore, the growing interest in AI-assisted healthcare technologies creates opportunities for integrating additional intelligent functionalities capable of improving operational decision-making processes.

External threats also represent important considerations for the project. Healthcare systems frequently operate under strict regulations related to privacy, security and compliance, which may introduce implementation challenges and additional development requirements. Since the platform handles medication and organizational information, sensitive medical data protection remains an important concern. Resistance to technology adoption among healthcare personnel may also slow implementation processes, particularly in environments where existing workflows rely heavily on traditional methods. Additionally, AI-assisted systems may generate incorrect predictions, potentially affecting inventory decisions if users rely excessively on automated recommendations without sufficient human oversight.

Overall, the SWOT analysis demonstrates that MediStock possesses several strong characteristics associated with simplicity, modularity and specialized inventory functionality. However, future development efforts should continue addressing AI reliability, data quality and operational adoption challenges in order to maximize long-term project impact.

Strenghts • Role-Based Access Control • Focused Scope • AI-assisted forecasting • Modular Architecture	Weaknesses • Limited historical data for AI • No hardware automation • Dependency on user manual input • AI accuracy uncertainty
Opportunities • Increasing demand for healthcare digitization • Expansion toward multi-hospital deployments • Advanced analytics and reporting • Growing interest in AI-assisted healthcare systems	Threats • Strict healthcare regulations • Sensitive medical data risks • User resistance to adoption • Incorrect AI predictions

Fig. 2. MediStock SWOT Analysis

3.4 FEASIBILITY STUDY

A feasibility study was performed to evaluate whether the MediStock platform could realistically be implemented from technical, operational and economic perspectives. The purpose of this analysis was to determine whether the proposed architecture, technologies and workflows are suitable for practical deployment and capable of supporting the intended project objectives.

From a technical perspective, the platform was considered feasible due to the use of modern technologies and modular design principles. The backend architecture relies on Spring Boot and REST APIs, allowing scalable communication and separation of business logic from presentation components. Authentication mechanisms are implemented using JWT-based stateless approaches, reducing server-side memory usage and simplifying scalability. Database integration also supports inventory entities, medication information and future expansion opportunities. On the frontend side, the system architecture enables role-based interfaces, dashboards and alert systems designed to provide users with operational visibility and simplified workflows.

The AI component was also evaluated from a feasibility perspective. The forecasting module introduces predictive capabilities focused on inventory planning and department-level medication analysis. By processing historical medication usage patterns, the AI module attempts to estimate future medication demand and support inventory planning activities. Future iterations may also incorporate expiration-aware analysis and additional predictive mechanisms intended to improve decision support processes. However, the forecasting component remains dependent on the availability and quality of historical information, which introduces limitations during early deployment stages.

From an operational perspective, the platform aligns with common healthcare workflows and organizational responsibilities. Users interact with the system according to predefined roles, while inventory activities such as medication management, stock updates and expiration monitoring closely resemble real-world operational procedures. The role-based structure contributes toward workflow separation and supports organizational clarity.

Economic feasibility was also considered during the analysis process. Compared to enterprise healthcare ecosystems such as Omnicell and Oracle Health, MediStock introduces significantly lower implementation and operational costs. The system does not require specialized hardware, robotics infrastructure or large-scale enterprise environments. As a result, deployment complexity and maintenance requirements remain lower, potentially making the platform more accessible to smaller hospitals and pilot healthcare environments.

Several risks and limitations were also identified. The forecasting component may suffer from limited AI training data and uncertainty regarding prediction accuracy. Manual data entry errors may negatively influence inventory reliability and forecasting outcomes. Security risks such as unauthorized access or sensitive medical data exposure remain important considerations due to the healthcare context of the platform. Additional risks include resistance to adoption among healthcare personnel and scalability limitations associated with future system growth.

Overall, the feasibility analysis indicates that MediStock represents a realistic and implementable solution capable of supporting medication inventory activities through a lightweight and modular architecture. While several technical and operational challenges remain, the proposed design demonstrates strong potential for future expansion and practical deployment.

4 SYSTEM REQUIREMENTS

This chapter describes the functional and operational requirements of the MediStock platform. System requirements define the expected behavior of the application and establish the conditions that must be satisfied in order to support medication inventory workflows and interactions between healthcare entities. These requirements serve as a foundation for implementation activities and ensure that both technical and business objectives remain aligned throughout the development process.

Unlike traditional inventory systems where users manually insert medications into a local inventory database, MediStock follows a transfer-oriented workflow. The system maintains a centralized medication list where inventory changes occur through transfer operations between hospitals, pharmacies and providers. Medication entities are therefore introduced into organizational inventories through controlled transfer procedures rather than through unrestricted manual creation processes. This approach improves traceability, reduces inconsistencies and allows better control over medication movement across the ecosystem.

Furthermore, transfer operations require explicit confirmation from both involved parties before modifications become effective. This requirement was introduced in order to preserve inventory consistency and ensure agreement between entities participating in medication exchanges. Transfer requests remain pending until both sides validate and approve the operation.

4.1 FUNCTIONAL REQUIREMENTS

The functional requirements define the expected behavior of the MediStock platform and describe the core functionalities necessary for supporting medication inventory workflows, transfer operations and system interactions. Since the platform follows a transfer-oriented inventory model, medication information is introduced and propagated through controlled exchanges between participating entities rather than unrestricted manual creation. The following requirements describe the major system functionalities identified during the analysis and planning stages.

4.1.1 FR-01 — Authentication and Authorization

The platform must authenticate users through JWT-based stateless mechanisms and provide access according to predefined user roles.

4.1.2 FR-02 — Role-Based Access Control

The system must restrict available actions according to the permissions associated with Hospital Manager, Pharmacist, Provider and Viewer roles.

4.1.3 FR-03 — Centralized Medication Visibility

Users must be able to access and visualize medication information through a centralized inventory interface.

4.1.4 FR-04 — Transfer-Based Inventory Management

Medication entities must not be introduced manually into organizational inventories. Inventory modifications shall occur through transfer operations between entities.

4.1.5 FR-05 — Transfer Request Creation

Authorized users must be able to initiate transfer requests between hospitals, pharmacies and providers.

4.1.6 FR-06 — Dual Confirmation Process

Medication transfers require approval from both participating parties before inventory updates become effective.

4.1.7 FR-07 — Automatic Inventory Synchronization

After transfer confirmation, inventory quantities must be updated automatically.

4.1.8 FR-08 — Expiration and Lot Tracking

The system must monitor medication expiration dates and associated lot information.

4.1.9 FR-09 — Dashboard and Alert System

The system must provide notifications regarding low stock levels and medications approaching expiration.

4.1.10 FR-10 — AI-Assisted Forecasting

The platform must analyze historical medication consumption, current stock levels, expiration dates and restock information in order to estimate future demand, identify shortage risks, detect expiration risks and generate AI-assisted transfer recommendations between healthcare institutions. AI-generated outputs must act as decision-support information and must not automatically modify inventory without user approval.

4.1.11 FR-11 — Transfer History and Logging

The system must maintain historical records of inventory modifications and transfer activities.

4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes and operational characteristics of the MediStock platform. Unlike functional requirements, which define what the system should do, non-functional requirements specify how the platform should behave under different conditions and establish expectations regarding usability, performance, scalability and security. These requirements are particularly important because they influence the overall user experience, system reliability and long-term maintainability of the application.

4.2.1 Scalability

The MediStock platform should be designed using a modular and scalable architecture capable of supporting future growth and additional functionalities without requiring major redesign efforts. Since the system is expected to evolve beyond its initial prototype scope, scalability represents an important consideration during development. Future versions may introduce support for multiple hospitals, broader provider interactions, larger inventory datasets and patient-oriented functionalities. The use of REST APIs, JWT-based stateless authentication and separated system components supports easier expansion and simplifies future deployment scenarios. The architecture should therefore be capable of accommodating increasing numbers of users, inventory transactions and AI processing activities while preserving system stability.

4.2.2 Usability

The platform should provide an intuitive and user-friendly interface capable of supporting healthcare personnel with varying levels of technical expertise. Since pharmacists, managers and other stakeholders interact with the system on a daily basis, usability directly affects operational efficiency and user adoption. The application should minimize unnecessary complexity and provide clear workflows for medication management, inventory monitoring and transfer operations. Dashboard components, alerts and inventory information should be presented in a simple and understandable manner in order to reduce user confusion and support faster decision-making processes.

4.2.3 Security

Security represents a critical requirement due to the sensitive nature of healthcare-related information and inventory operations. The platform should ensure that users access functionalities according to predefined permissions and organizational responsibilities. Authentication mechanisms should rely on JWT-based stateless approaches and authorization processes should enforce role-based restrictions. Sensitive information and critical operations such as transfers, inventory modifications and approval actions should remain protected against unauthorized access. Additionally, the system should support secure communication mechanisms and maintain operational logs in order to improve traceability and accountability.

4.2.4 Performance

The MediStock platform should provide responsive interactions and maintain acceptable response times during daily operational usage. Inventory visualization, dashboard loading, medication searches and transfer workflows should execute efficiently in order to avoid disrupting

healthcare processes. Since operational decisions may depend on immediate access to inventory information, delays or excessive processing times could negatively affect user experience and workflow effectiveness. The AI forecasting component should also generate predictions within reasonable time limits without introducing significant delays in system interactions. Overall, the system should maintain stable performance under normal operational workloads while supporting future growth and additional functionalities.

4.3 BUSINESS RULES

Business rules define the operational constraints, policies and domain-specific conditions that govern the behavior of the MediStock platform. These rules establish how the system should behave in healthcare-related scenarios and ensure that inventory processes remain consistent, reliable and aligned with real-world operational practices. Unlike functional requirements, business rules describe organizational logic and restrictions that guide decision-making and system workflows.

Since MediStock operates in a healthcare inventory environment, business rules are particularly important because medication handling directly influences operational efficiency and inventory integrity. The following rules were identified during the analysis and planning phases and represent essential constraints that must be respected throughout the system lifecycle.

4.3.1 BR-01 — Expired medications cannot be actively used or transferred.

The system must prevent medications that have reached their expiration date from being considered available inventory. Expired products should be identified automatically and excluded from operational workflows in order to reduce risks and medication waste.

4.3.2 BR-02 — Medication inventory updates must occur through transfer operations.

The platform follows a transfer-oriented inventory model. Medication records are not introduced manually into organizational inventories. Inventory changes occur only through validated transfer workflows between participating entities.

4.3.3 BR-03 — Transfer requests require approval from both parties.

Medication transfers between hospitals, pharmacies or providers must remain in a pending state until both involved entities explicitly confirm the operation. Inventory updates become effective only after dual confirmation has been completed.

4.3.4 BR-04 — Stock quantities cannot become negative.

The system must prevent inventory operations that would result in negative medication quantities. Validation mechanisms should ensure that available stock remains greater than or equal to zero.

4.3.5 BR-05 — Medication transfers cannot exceed available inventory.

Users cannot initiate transfer requests for quantities larger than the currently available stock. Transfer validation procedures should verify inventory availability before requests become active.

4.3.6 BR-06 — Users can only perform actions permitted by their assigned role.

System functionality must follow predefined role-based permissions. Hospital Managers, Pharmacists, Providers and Viewers should only access actions corresponding to their organizational responsibilities.

4.3.7 BR-07 — Inventory modifications must be traceable.

Critical actions such as stock updates, transfer confirmations and inventory modifications should generate records that can later be reviewed for auditing and operational analysis purposes.

4.3.8 BR-08 — AI recommendations act as decision-support mechanisms only.

AI-generated predictions and transfer recommendations must not automatically trigger inventory changes. The AI module may identify shortage risks, expiration risks and potential transfer opportunities, but the final decision remains under human supervision. Transfer recommendations generated by the AI are stored with a pending status and require user action before any operational workflow continues.

4.3.9 BR-09 — Medication lot information must remain associated with inventory records.

Inventory entities should preserve lot-related information including batch numbers, expiration dates and supplier associations. This rule supports traceability and improves expiration monitoring capabilities.

4.3.10 BR-10 — Inventory information must remain synchronized after transfer completion.

Following successful transfer validation and approval, inventory quantities should automatically update for both sending and receiving entities in order to maintain consistency across the platform.

These business rules establish the operational foundation of MedStock and ensure that inventory workflows remain controlled, transparent and aligned with real healthcare practices. They also contribute toward reducing inconsistencies and improving medication lifecycle management throughout the system.

5 SYSTEM DESIGN & ARCHITECTURE

This chapter describes the architectural design and major system components used in the development of the MediStock platform. The architecture was designed following modular development principles in order to maintain separation between presentation, business logic, artificial intelligence functionalities and data persistence mechanisms. The objective of this approach is to improve maintainability, scalability and future extensibility while ensuring that individual components can evolve independently.

The MediStock platform follows a layered architecture in which the frontend, backend, AI module and database layer communicate through clearly defined interfaces. Each component has a specific responsibility and contributes to the overall workflow of the system. This separation reduces implementation complexity and enables easier maintenance and integration of future functionalities.

5.1 HIGH-LEVEL ARCHITECTURE

The MediStock platform is structured around four major architectural components consisting of the frontend layer, backend services, artificial intelligence module and database infrastructure. Together, these components form a complete ecosystem responsible for handling user interactions, business processes, predictive functionalities and persistent storage.

The frontend layer was implemented using Angular, which serves as the presentation component of the system and acts as the primary interaction point for users. Angular was selected due to its component-based architecture and support for modular development practices. Through Angular, users can access inventory dashboards, medication information, transfer workflows and role-based functionalities. The frontend is responsible for displaying data, validating user input and communicating with backend services through REST API requests. Additionally, the user interface was designed to support role-specific interactions and provide operational visibility through dashboards and alert systems.

The backend layer was implemented using Spring Boot and represents the core business logic component of the platform. Spring Boot is responsible for processing requests received from the frontend, validating operations and managing inventory workflows. This component handles authentication, role-based authorization, transfer operations, inventory synchronization and communication with external modules. The backend also implements a stateless architecture using JWT authentication mechanisms in order to improve scalability and reduce memory consumption. Since business rules and operational logic are centralized within this layer, it serves as the primary coordination point of the system.

The artificial intelligence component was implemented as a separate Python-based analytical engine responsible for forecasting medication demand, identifying inventory risks and generating transfer recommendations. The AI module does not rely on large language models or external AI services. Instead, it uses transparent analytical logic, weighted historical consumption averages, rule-based risk classification and Monte Carlo simulations for confidence estimation.

The system uses confidence scores at two levels. First, prediction confidence measures how stable a shortage or expiration risk remains under Monte Carlo demand simulation. Second, recommendation confidence evaluates the reliability of a proposed transfer by combining prediction confidence with logistics feasibility, financial margin and destination consumption capacity.

The AI workflow is divided into two main stages. The first stage runs the analysis engine, which calculates predicted daily demand, shortage risks, expiration risks, alerts and confidence scores. The second stage runs the recommendation engine, which matches hospitals with expiration-risk batches to hospitals with shortage risks for the same medication. Before generating a recommendation, the system validates storage compatibility, transport feasibility, source safety stock, destination need, expected consumption before expiration and positive financial impact.

The AI engine is exposed through a dedicated FastAPI service. This service can be triggered manually through API endpoints or automatically when database changes are detected. When an AI refresh is executed, the service first runs the risk analysis script and then the transfer recommendation script. The generated results are persisted in PostgreSQL and later consumed by the backend and frontend through database views.

To avoid unnecessary repeated processing, the system uses an AI refresh state mechanism. Operational database changes mark the AI results as outdated, while the AI service periodically checks whether a refresh is required. The service also prevents overlapping AI executions by using locking mechanisms, ensuring that two AI runs are not executed simultaneously.

The persistence layer of the platform was implemented using Supabase, which acts as the primary database infrastructure responsible for storing and managing system information. Supabase was selected due to its support for modern application architectures and simplified

integration capabilities. The database stores information related to medications, transfer operations, inventory states, user roles and operational records. Additionally, the database maintains historical information necessary for tracking activities and supporting future analytical or forecasting functionalities.

The communication flow inside the system begins with user interaction through the Angular frontend. User requests are transmitted through REST APIs toward the Spring Boot backend, where business logic and validation mechanisms are executed. The backend interacts with the Supabase database for data retrieval and storage operations while also communicating with the AI module whenever predictive analysis is required. The results generated by these components are then returned to the frontend and presented to users through role-specific interfaces.

The database layer uses a normalized PostgreSQL structure hosted through Supabase. Instead of storing all medication information in a single table, the persistence layer separates medication catalogue data, batch-level stock, hospital ownership, transfer workflows, alerts, audit logs and AI-related outputs. This structure allows the backend and AI modules to operate on accurate inventory data while preserving traceability across hospitals, batches, transfers and recommendations.

This architectural structure promotes modularity and simplifies future development activities while preserving the flexibility required for integrating new features and supporting larger deployment scenarios.

5.2 STATELESS ARCHITECTURE

The MediStock platform adopts a **stateless architectural** approach combined with **JWT-based authentication** mechanism in order to improve scalability, simplify request processing and reduce server-side complexity. Since the platform is expected to support multiple users interacting simultaneously with inventory operations, transfer workflows and AI-assisted functionalities, selecting an appropriate authentication and session management strategy represented an important architectural decision during the system design phase.

Traditional stateful systems commonly rely on server-side sessions in which the server stores information regarding authenticated users directly in memory. In this approach, the server maintains active session information and uses it to identify users during subsequent interactions. While this model can simplify authentication logic, it introduces several limitations associated with scalability, memory usage and fault tolerance. Maintaining user sessions on the server requires additional resources and may complicate deployment scenarios involving multiple application instances or load balancing mechanisms.

To avoid these limitations, MediStock implements a stateless communication model. In a stateless architecture, the server does not preserve user-specific session information between requests. Instead, every request is treated independently and contains all information required for processing and authentication. This design ensures that the backend remains lightweight and eliminates the need for maintaining active user sessions directly on the server.

Authentication is implemented through JSON Web Tokens (JWT). After a successful login process, the backend validates user credentials and generates a token containing authentication and authorization information associated with the user. This token is then stored on the client side and included in subsequent requests sent toward the backend. During each request, the server validates the received token and determines whether the user has sufficient permissions to perform the requested operation.

The use of JWT offers several advantages for the MediStock platform. Since authentication data is transferred together with every request, the backend does not need to allocate additional memory resources for session management. This reduces server-side overhead and simplifies system behavior. Additionally, the architecture becomes more scalable because multiple backend instances can process requests independently without requiring synchronized session information.

Another important advantage of the stateless approach involves fault tolerance and maintainability. If a backend instance restarts or becomes unavailable, user interactions remain less affected because authentication information is preserved on the client side rather than being stored in server memory. This behavior supports more resilient deployment scenarios and facilitates future scaling strategies.

The stateless model also aligns naturally with REST architectural principles, since REST APIs are designed around independent requests and resource-oriented interactions. Because MediStock relies heavily on REST communication between frontend and backend components, JWT-based stateless authentication integrates naturally into the overall system design.

Although stateless architectures provide multiple advantages, certain limitations must also be considered. Since authentication information is stored within tokens, secure handling and storage of JWT values become critical. Additionally, token invalidation and session termination mechanisms may require additional implementation considerations compared to traditional server-side sessions.

Overall, the decision to implement JWT-based stateless authentication was motivated by lower memory usage, simplified scalability, improved fault tolerance and better alignment with modern web application development practices. This architectural approach supports the lightweight and modular objectives of MediStock while preserving flexibility for future system expansion.

5.3 USER WORKFLOW – MEDICATION

The medication workflow within MediStock was designed around a transfer-based inventory model rather than traditional manual medication insertion processes. Unlike conventional inventory systems where users directly add medications into local stock repositories, the proposed workflow introduces medications through controlled transfer procedures between participating healthcare entities. This approach improves traceability, reduces inventory inconsistencies and ensures that medication movements remain synchronized across the ecosystem.

The workflow begins when a hospital or pharmacy identifies the need for additional medication inventory. In the standard process, a transfer request is initiated by one organization and sent to another participating entity. The request contains relevant information associated with the medication, including transfer quantities and inventory details. Once the request is generated, the receiving organization reviews the request and decides whether the transfer can be fulfilled.

To preserve inventory consistency and avoid unilateral modifications, MediStock introduces a dual confirmation mechanism. Transfer operations remain in a pending state until the receiving institution reviews the request and either accepts or rejects it. Inventory changes become effective only after the transfer request is accepted by the authorized receiving institution.

The receiving side validates the request and confirms availability, while the initiating side also confirms the operation before final execution. Until the request is accepted, the transfer remains in a pending state and no stock movement is performed. This mechanism reduces inconsistencies and ensures agreement between all involved parties.

Following successful approval, the inventory synchronization process is automatically triggered. The medication list and inventory quantities are updated accordingly, ensuring that both organizations maintain accurate stock visibility. The system records transfer history and inventory modifications in order to support traceability and future auditing activities.

The workflow also supports an AI-assisted transfer mechanism intended to improve decision-making processes. Instead of relying exclusively on manual inventory analysis, the AI component evaluates available stock information, historical transfer activity and inventory conditions to generate recommendations regarding potentially beneficial transfer opportunities.

Through predictive analysis, the system may suggest inventory movements between hospitals based on estimated demand or anticipated shortages. Users still retain control over final decisions, as AI recommendations act strictly as decision-support mechanisms rather than automated actions.

The workflow therefore operates in two distinct scenarios. The first scenario follows a traditional transfer process where users manually initiate inventory movement requests. The second scenario introduces an AI-assisted recommendation layer capable of identifying optimal transfer opportunities and supporting proactive inventory planning. Both approaches ultimately follow the same validation and approval mechanisms, ensuring consistency and preserving organizational control over medication movement operations.

This workflow structure combines operational control, inventory transparency and predictive decision support while maintaining a lightweight and scalable inventory management process.

6 BACKEND DEVELOPMENT

The backend component of MediStock represents the core layer responsible for business logic, data validation, authentication, authorization and communication with the database. The backend was implemented using Spring Boot and follows a structured three-tier architecture composed of Controller, Service and Repository layers. Controllers handle incoming HTTP requests, Services encapsulate business logic and validation rules, while Repository components manage database persistence operations. This separation improves maintainability by ensuring that HTTP request handling, business rules and data persistence logic are organized into distinct components. As a result, the backend becomes easier to extend, debug and test as the platform grows.

6.1 API CONTRACT

The communication between the frontend and backend is performed through REST APIs. Although OpenAPI or Swagger were not integrated in the current development phase, the API contract was manually defined and aligned between frontend and backend responsibilities. This contract describes the available endpoints, expected request structures, response formats and error handling behavior. The backend exposes endpoints for medication management, transfer requests, authentication, inbox messages, filtering and audit-related operations. CORS was configured in order to allow communication between the Angular frontend running on port 4200 and the Spring Boot backend running on port 8080. The configuration allows standard HTTP methods used for CRUD operations, accepts required headers and supports cookie-based communication between the frontend and backend.

6.2 ENTITY DESIGN

The backend data model was designed using a normalized database structure in order to support realistic medication inventory workflows, transfer operations and AI-assisted analysis. Unlike earlier simplified approaches where inventory information was stored directly inside a single medication table, the current implementation separates medication definitions from physical stock availability. This architecture improves scalability, avoids data duplication and enables more accurate inventory tracking across hospitals and institutions.

In addition to database entities, the backend implementation uses Data Transfer Objects (DTOs) to separate internal persistence models from externally exposed API structures. DTOs act as intermediary objects used during communication between backend and frontend components. The use of DTOs improves maintainability and security by preventing direct exposure of database entities while reducing tight coupling between persistence models and external interfaces. DTOs are also used for request validation, response formatting and transferring structured information between Controller and Service layers.

The medication system is divided into two major entities: the Medication catalog and Medication batches.

The medications table represents the general medication catalogue and stores static descriptive information associated with medication definitions. Unlike physical inventory records, this entity defines what a medication is rather than where and how much of it exists. The entity contains attributes such as medication name, internal code, pharmaceutical form, therapeutic class, storage type and criticality level. This approach prevents duplicated medication definitions and allows multiple hospitals to reference the same medication information.

Physical inventory information is managed through the medication_batches table. This entity stores the actual medication stock available in healthcare institutions. Each batch is associated with a specific medication and hospital through identifiers such as medicationId and hospitalId. Additional fields include batchNumber, quantityCurrent and expiryDate. This design enables multiple batches of the same medication to coexist simultaneously with different expiration dates and quantities. Batch-level management is particularly important because hospitals frequently receive medication deliveries from different suppliers and at different times.

By separating inventory into batches, the system can more accurately support expiration-aware analysis, transfer recommendations and stock monitoring processes. Since AI logic relies heavily on expiration dates and quantity information, batch-level granularity significantly improves prediction quality.

To simplify frontend visualization and inventory access, the application uses a PostgreSQL view named v_medication_availability_by_institution, represented in the backend through the MedicationAvailabilityView entity. This view combines medication catalogue information with batch availability and institution-level inventory details. Rather than requiring the frontend to manually combine information from several entities, the view exposes a unified representation of medication availability.

User information is managed through the app_users table and mapped using the User entity. The current implementation includes fields such as id, hospitalId, fullName, username, password, email, role, isActive, createdAt and updatedAt. The hospitalId field creates a relationship between users and healthcare institutions, enabling hospital-specific ownership and access control logic. The role attribute defines authorization responsibilities and permissions associated with users such as Hospital Managers, Pharmacists and Providers.

Several additional entities support operational workflows and AI functionality. The hospitals table stores institution-level information and organizational metadata. The departments entity supports internal hospital structures and department-specific AI forecasting. Medication

movement operations are managed through `transfer_requests`, while AI-generated redistribution suggestions are stored inside `transfer_recommendations`. Risk notifications and warning messages are managed through the `alerts` table, while `audit_logs` stores critical operational events for traceability and monitoring purposes. Communication-related features are implemented through `inbox_messages`, which supports read and unread message states. Historical inventory behavior used by the AI module is stored in `daily_consumption`, and transportation constraints between institutions are represented through `transport_routes`.

The relationship model of the system extends beyond simple ownership associations. Hospitals may contain multiple users, medications can have multiple batches and transfer operations can involve multiple institutions simultaneously. This normalized design provides greater flexibility and supports more realistic healthcare inventory scenarios.

Overall, the current entity architecture provides a significantly more scalable and realistic foundation for the MedStock platform. By separating medication catalogues, inventory batches, users, transfer workflows and AI-related entities, the system supports accurate inventory tracking, expiration monitoring and predictive decision support while preserving maintainability and future extensibility.

6.3 AUTHENTICATION

Authentication in the backend is implemented using JWT-based stateless mechanisms. After successful authentication, the generated JWT token is not returned directly to the frontend application for manual storage. Instead, the token is stored inside secure HTTP cookies and automatically included in subsequent requests sent by the browser. This approach improves security by reducing direct JavaScript access to authentication tokens and minimizing risks associated with token exposure or client-side manipulation.

The backend validates the token received through cookies during every request and extracts the authenticated username and role information required for authorization and ownership-related operations.

The authentication mechanism also supports role-based access control. Users can have different permissions depending on their assigned role. For example, administrative users may have broader access rights, while other users may be restricted to specific operations. This role-based approach ensures that sensitive operations are only available to authorized users and that each stakeholder interacts with the platform according to their responsibilities.

The backend also includes security-related improvements such as hiding sensitive password fields from API responses and validating user existence before applying ownership logic. These mechanisms contribute to safer data handling and reduce the risk of exposing sensitive authentication information through normal API operations.

6.4 BUSINESS LOGIC

The backend business logic includes several operational rules that support medication management, transfer handling, filtering, validation and error management. One important implemented feature is the logic for rejecting transfer requests. This allows transfer workflows to support both acceptance and rejection scenarios, making the system more realistic and aligned with actual operational processes between hospitals, pharmacies and providers.

Filtering functionality was also implemented for medication records. Users can filter medications based on name, including partial name matches, category and status represented

through active or inactive states. This improves usability by allowing users to quickly locate relevant medication information within the system.

Several validation mechanisms were implemented in order to improve data quality. The backend resolves issues related to negative stock and price values, ensuring that invalid medication records cannot be created or updated. Validation was also added for received dates and expiration dates, reducing inconsistencies in medication lifecycle information. Additionally, previous status-related CRUD logic was removed from the Medication entity and database structure, since medication status and stock status are now handled inside the medication view layer.

Global exception handling was extended in order to manage backend errors in a consistent way. This ensures that validation problems, invalid requests and operational errors return predictable responses to the frontend. Standardized error handling improves debugging, user feedback and frontend-backend integration.

Overall, the backend development phase established the technical foundation for MediStock by implementing a structured architecture, entity relationships, authentication mechanisms, transfer request logic, filtering capabilities, CORS configuration, audit logging and validation rules. These features support the core inventory workflows of the platform and provide a stable base for future frontend, AI and reporting integrations.

7 FRONTEND

The frontend component of MediStock was developed as the presentation layer of the platform and serves as the primary interaction point between users and the system. The frontend was implemented using Angular, allowing the development team to create a modular and component-oriented interface capable of supporting inventory workflows, transfer operations and role-based interactions. The main objective of the frontend design was to provide users with an intuitive environment where operational tasks could be completed efficiently while preserving visibility over medication-related information and inventory activities.

The frontend architecture follows a component-based structure in which each visual element and workflow is implemented independently and later integrated into larger operational pages. This approach improves maintainability and enables future scalability as new functionalities are introduced. Since different users interact with the platform according to different responsibilities, the frontend also supports role-oriented workflows and dynamically presents information depending on the authenticated user.

7.1 UI DESIGN

The user interface was designed with simplicity and operational efficiency in mind. Since healthcare environments frequently require fast interactions and immediate access to important information, the interface focuses on reducing unnecessary complexity while maintaining visibility over inventory operations.

The overall interface structure includes dashboard pages, medication visibility screens, transfer workflows and operational notifications. Navigation was designed to support straightforward access to important system areas while minimizing the number of steps required for completing common tasks.

Special attention was given to inventory visibility and transfer-related interactions. Since MediStock follows a transfer-oriented model, the user interface was adapted to reflect medication movement processes rather than traditional manual inventory insertion workflows. Instead of encouraging direct creation of medication records through inventory pages, users interact primarily with transfer requests and inventory synchronization activities.

Visual elements such as tables, filters, notifications and status indicators were integrated in order to improve operational awareness and help users identify important inventory events quickly.

7.2 COMPONENTS

The frontend implementation consists of multiple reusable components responsible for displaying information and supporting user interactions throughout the system.

One of the main components is the Medication List component. This section allows users to visualize available medication information and inventory-related details. Medication entries display information such as medication names, categories, stock quantities, suppliers and expiration data. Since the platform no longer follows a direct medication creation model, users do not manually insert medication records through this interface. Instead, medications become visible inside the inventory after successful transfer operations between participating entities. Therefore, the medication list acts primarily as an inventory visibility component rather than a traditional CRUD interface.

Filtering functionality was also implemented to improve navigation and information retrieval. Users can search medications by name, including partial matches, as well as filter records according to category and status information. These features improve usability and simplify inventory exploration when larger datasets are involved.

The frontend also includes various Form Components responsible for handling structured user input. Unlike earlier system iterations where medication forms were intended for direct medication creation, forms are now mainly used for operational workflows such as authentication processes, transfer requests and future inventory-related interactions. These components validate user input and communicate with backend services before submitting requests.

Another important frontend component is the Dashboard interface. The dashboard serves as a centralized operational overview where users can access summaries related to inventory activity and system information. Dashboard components may display alerts, inventory statistics and operational notifications intended to improve visibility and support decision-making activities.

As the platform evolves, additional components may be integrated in order to support AI forecasting visualizations, transfer history tracking and advanced reporting functionalities.

7.3 FRONTEND—BACKEND COMMUNICATION

Communication between the frontend and backend components is performed through REST APIs. Angular components generate HTTP requests that interact with Spring Boot endpoints in order to retrieve, update and synchronize information across the system.

Since the frontend and backend operate on different ports during development, Cross-Origin Resource Sharing (CORS) was implemented to allow communication between the Angular application running on port 4200 and the backend services running on port 8080. The CORS configuration allows standard HTTP methods and supports secure communication between both application layers.

The frontend consumes backend endpoints responsible for authentication, medication visibility, transfer requests, filtering functionalities and operational workflows. Data exchanged between frontend and backend components follows JSON structures transmitted through REST-based interactions.

Authentication communication follows a JWT-based workflow. After successful login, authentication information is transmitted to the backend and a token is generated for subsequent requests. This mechanism allows the frontend to maintain authenticated sessions while preserving the stateless architecture of the platform.

Overall, the frontend–backend communication structure ensures separation between presentation logic and business logic while supporting scalability and future integration opportunities.

8 AI DEVELOPMENT

The artificial intelligence component of MediStock was introduced in order to support intelligent decision-making and improve medication inventory planning processes. Unlike traditional inventory systems that rely only on fixed thresholds and manual analysis, the AI module analyzes historical consumption, current stock levels, expiration dates, restock information and logistics constraints in order to identify operational risks and recommend possible stock redistribution actions.

The implemented AI workflow does not rely on large language models, external AI services or black-box machine learning models. Instead, it uses a custom Python-based analytical engine composed of weighted forecasting formulas, rule-based risk classification, Monte Carlo uncertainty simulation and explainable recommendation logic. This approach was selected because it provides transparent results, predictable behavior and direct control over the generated predictions and recommendations. The AI module does not automatically create procurement or restock orders. Existing restock orders are used as input when calculating effective stock coverage, but procurement actions remain outside the automated AI workflow.

Each AI execution refreshes the current prediction and recommendation outputs. Previous AI-generated predictions, alerts and recommendations are cleared before new results are inserted, ensuring that the frontend displays the latest calculated state of the system.

8.1 AI OBJECTIVE

The primary objective of the AI component is to predict future medication demand, estimate inventory risks and recommend transfer operations between healthcare entities when inventory imbalances are detected. Rather than performing isolated predictions, the module combines multiple analytical stages in order to generate decision-support recommendations.

The first implemented functionality estimates future daily medication demand. This prediction uses weighted historical consumption patterns based on different observation periods. The implemented approach assumes that recent behavior should influence predictions more strongly than older data while still preserving longer-term trends.

The daily demand prediction is calculated using the following formula:

$$\text{predicted daily demand} = 0.50 * (AVG_{CONSUMPTION_{7D}}) + 0.30 * (AVG_{CONSUMPTION_{30D}}) + 0.20 * (AVG_{CONSUMPTION_{90D}})$$

The formula gives greater importance to the most recent seven days of activity while also considering monthly and longer-term historical behavior.

After demand prediction, the AI module estimates stock shortage risk by evaluating inventory coverage relative to predicted demand. Inventory coverage represents the estimated number of days during which the current stock remains sufficient.

The initial stock coverage calculation follows:

$$\text{raw stock coverage days} = \frac{\text{current stock}}{\text{predicted daily demand}}$$

If incoming restock operations exist and arrive before inventory depletion occurs, the system adjusts available inventory and recalculates effective stock coverage.

Based on resulting coverage values, the system classifies inventory conditions into several risk categories ranging from high risk to stable operational states. This mechanism allows users to identify potential shortages before inventory becomes critical.

The AI component also predicts expiration-related risks. This functionality estimates whether current medication quantities can realistically be consumed before reaching expiration dates.

Days remaining before expiration are calculated using:

$$\text{days to expiry} = \text{expiry date} - \text{today date}$$

Estimated future consumption until expiration is then determined:

$$\text{predicted_consumption_until_expiry} = \text{predicted_daily_demand} \times \text{days_to_expiry}$$

The quantity likely to expire is estimated as:

$$\text{estimated_expired_quantity} = \max(0, \text{quantity_current} - \text{predicted_consumption_until_expiry})$$

Based on these values, the AI component determines risk levels associated with expiration events and identifies medications requiring intervention.

The final objective of the AI module involves generating transfer recommendations between healthcare entities. This stage combines inventory risk, expiration analysis and operational constraints to recommend inventory movement opportunities.

Transfer recommendations are not generated solely based on predicted expiration quantities. The system additionally considers safety stock requirements, destination inventory needs, transportation constraints, storage compatibility and expected economic benefits.

The recommended transfer quantity is determined using:

$$\text{recommended_quantity} = \min(\text{estimated_expired_quantity}, \text{source_transferable_quantity}, \text{destination_need})$$

Recommendations become valid only if several operational conditions are satisfied, including storage compatibility, transport feasibility, positive financial impact and sufficient consumption capacity at destination locations.

The source transferable quantity is calculated by keeping a safety stock at the source hospital:

$$\text{source_safety_stock} = \text{predicted_daily_demand_source} \times \text{min_buffer_days_source}$$
$$\text{source_transferable_quantity} = \max(0, \text{quantity_current} - \text{source_safety_stock})$$

The destination need is calculated based on the target stock level required by the destination hospital:

$$\text{target_stock_destination} = \text{predicted_daily_demand_destination} \times \text{target_buffer_days_destination}$$
$$\text{destination_need} = \max(0, \text{target_stock_destination} - \text{current_stock_destination} - \text{incoming_restock_quantity})$$

The financial impact of a recommendation is calculated using the estimated value of the medication saved from expiration, the avoided disposal cost and the estimated transport cost:

$$\text{saved_medication_value} = \text{recommended_quantity} \times \text{purchase_price_per_unit}$$
$$\text{avoided_disposal_cost} = \text{recommended_quantity} \times \text{disposal_cost_per_unit}$$
$$\text{net_savings} = \text{saved_medication_value} + \text{avoided_disposal_cost} - \text{transport_cost}$$

Before a transfer recommendation is stored by the recommendation engine, several operational constraints are validated in order to ensure that the proposed transfer is feasible, safe and beneficial for both participating institutions. The system first verifies that the source and destination institutions are different entities, preventing invalid self-transfer scenarios. It then checks whether the destination institution possesses compatible and validated storage conditions required for the specific medication.

Additionally, the recommendation engine evaluates the existence of a valid transport route between both institutions and confirms that the selected route supports the necessary transportation requirements, such as cold chain, frozen chain or controlled transport conditions. The system further ensures that the source hospital maintains inventory above its minimum safety stock threshold after the transfer is completed, preventing the recommendation from creating shortages at the source location. Simultaneously, the destination institution must demonstrate an actual shortage-related need for the same medication. The engine also estimates whether the destination can realistically consume the transferred quantity before the medication batch reaches its expiration date. Furthermore, the proposed transfer quantity must exceed a predefined minimum recommendation threshold to avoid operationally insignificant transfers. Finally, the recommendation is accepted only if the estimated net financial savings remain positive, ensuring that the transfer creates measurable operational and economic value.

The recommendation confidence score is calculated separately from the risk confidence scores. It combines the reliability of the source expiration prediction, the reliability of the destination shortage prediction, the logistics feasibility of the route, the financial savings margin and the destination's ability to consume the recommended quantity before expiration. This means that a recommendation with high expiry and shortage risk may still receive a lower confidence score if the route is close to the maximum allowed distance, the savings margin is small or the destination may not consume the quantity in time. The recommendation engine only considers relevant HIGH and MID risk situations. Low-risk or stable inventory conditions are not used as primary sources for transfer recommendation generation.

A recommendation is considered valid only if the estimated net savings are positive. The quantity likely to expire is estimated as:

$$\text{estimated_expired_quantity} = \max(0, \text{quantity_current} - \text{predicted_consumption_until_expiry})$$

The expiration risk level is classified according to the number of days remaining until expiration and the estimated expired quantity:

HIGH — the batch is expired or will expire within 30 days and still has estimated remaining quantity. MID — the batch will expire within 90 days and still has estimated remaining quantity. LOW — the batch has lower expiration urgency. OK — no relevant quantity is estimated to expire

8.1.1 Monte Carlo Confidence Estimation

In addition to deterministic risk classification, the AI module calculates a confidence score using Monte Carlo simulation. This mechanism is used to estimate how stable a predicted risk level remains when future demand varies realistically.

For each hospital–medication pair or medication batch, the system analyzes historical daily consumption and calculates demand uncertainty using the historical mean, standard deviation and number of available consumption days. Based on this uncertainty, the system generates multiple plausible future demand values. In the current implementation, 250 simulations are executed for each risk evaluation.

During each simulation, the system recalculates the relevant risk level. For shortage risk, it recalculates the stock coverage using the simulated demand. For expiration risk, it recalculates the estimated quantity that may remain unused before the batch expires. The final confidence score is based on how many simulations produce the same risk level as the original prediction.

A high confidence score means that the predicted risk remains stable under realistic demand variation. A lower confidence score means that the risk classification is more uncertain and should be interpreted with additional caution.

8.1.2 AI Outputs

The AI module stores its results in dedicated database tables. Risk predictions are stored in the `ai_predictions` table and include prediction type, risk level, predicted daily demand, stock coverage, days to expiry, estimated expired quantity, confidence score and an input snapshot used for explainability. Alerts are stored in the `alerts` table and are generated for relevant HIGH or MID risk situations.

Transfer recommendations are stored separately in the `transfer_recommendations` table. Each recommendation contains the source institution, destination institution, medication, batch, recommended quantity, risk level, expected savings, avoided disposal cost, transport cost, net savings, distance, recommended transfer date, confidence score and status. Recommendations are initially created with a pending status and require user action before they can influence operational workflows.

8.1.3 AI Execution Workflow

The AI execution workflow is divided into two scripts and one orchestration service.

The first script, `run_analysis.py`, performs the main risk analysis. It creates a new AI run log, calculates medication demand forecasts, evaluates shortage risks, evaluates expiration risks, calculates Monte Carlo confidence scores and stores the generated predictions and alerts.

The second script, `generate_transfer_recommendations.py`, uses the latest successful AI run and reads the generated HIGH and MID shortage and expiration risks. It then attempts to match expiration-risk batches from one institution with shortage-risk needs from another institution. Valid recommendations are saved only after storage, route, safety stock, destination need, consumption capacity and financial constraints are verified.

The AI service orchestrates these scripts. It can be executed manually for testing or demonstration purposes, or automatically when operational database changes mark the AI results as outdated.

8.2 DATASET

The dataset used during AI development was artificially generated and represents a simulated environment designed specifically for validating inventory workflows and recommendation behavior. Since publicly available datasets containing detailed medication inventory transfers, expiration information and multi-hospital operational interactions were not identified during the research phase, a synthetic dataset was created to support experimentation and development activities.

The generated dataset includes medication records, historical consumption values, expiration information, inventory quantities, transportation conditions and hospital-related operational attributes. These values were designed to simulate realistic medication inventory scenarios and allow the recommendation engine to process situations involving shortages, expiration risks and transfer opportunities.

Although synthetic datasets provide flexibility and support rapid experimentation, they also introduce limitations. Because generated data may not fully capture real-world hospital behavior, prediction quality and recommendation accuracy may differ when applied in operational environments. Future development stages may require access to larger real-world datasets in order to improve prediction reliability, calibrate analytical thresholds and validate the recommendation logic against real hospital behavior.

The synthetic dataset was generated to include hospitals, pharmacies, medication batches, historical consumption records, restock orders, transport routes, storage capabilities and expiration scenarios. This structure allows the AI module to evaluate realistic operational situations such as shortage risk, expiration risk, redistribution opportunities and financial savings.

8.3 EXPLAINABILITY

A major objective during AI development was preserving transparency and avoiding black-box decision-making behavior. Rather than presenting users with unexplained recommendations, the MedStock AI module provides explicit reasoning describing why a recommendation was generated.

The explainability mechanism was implemented directly within the recommendation engine and does not rely on LLM technologies. Instead, the system constructs detailed reasoning statements based on analytical outputs generated during previous stages.

For example, recommendation explanations include information regarding source inventory conditions, estimated expiration quantities, destination shortage risks, stock coverage duration, transportation constraints and expected economic savings.

A recommendation may explain that a transfer is suggested because a specific hospital currently possesses inventory quantities at risk of expiration while another hospital simultaneously exhibits high shortage risk and can consume the recommended quantity before expiration occurs.

The reasoning output also incorporates transportation distance, storage compatibility requirements and estimated financial impact values such as net savings and avoided disposal costs. Confidence scores associated with recommendations are additionally displayed in order to provide users with visibility regarding recommendation reliability.

This explainability mechanism improves transparency and user trust while allowing healthcare personnel to understand the factors influencing AI-generated recommendations. Instead of replacing human decisions, the AI component acts as a decision-support system that provides contextual reasoning and preserves operational control within healthcare workflows.

In the implemented system, explanations are stored together with AI outputs. Risk explanations are saved in the prediction records, while transfer recommendation explanations are saved in the recommendation reason field. This allows the frontend to display not only the recommendation itself, but also the operational reasoning behind it, including batch information, stock risk, destination need, distance, storage requirements, estimated net savings and confidence score.

8.4 AI TESTING

The AI component was validated by executing the analysis workflow on the generated synthetic dataset and verifying that predictions, alerts and transfer recommendations were created correctly. The validation focused on checking whether the system generated shortage risks for institutions with low stock coverage, expiration risks for batches close to expiration and transfer recommendations only when operational constraints were satisfied.

The AI testing process also verified that recommendation generation respected storage compatibility, route feasibility, positive net savings and destination consumption capacity before expiration. The confidence scores were reviewed to ensure that they were generated through Monte Carlo simulation rather than fixed values.

Additional validation was performed by checking the database output tables after each AI run. The `ai_predictions` table was used to verify shortage and expiration risk outputs, the `alerts` table was used to verify generated warnings and the `transfer_recommendations` table was used to verify proposed redistribution opportunities.

A specific integration test should verify that after a transfer is accepted and inventory quantities are updated, the AI refresh mechanism detects the database change and regenerates predictions and transfer recommendations. The expected result is that recommendations no longer valid after the stock movement are removed from the current AI output.

9 TESTING AND VALIDATION

The testing and validation phase was performed in order to verify whether the implemented Medication API behaves correctly under both valid and invalid input conditions. The QA process focused mainly on functional testing and API testing for medication-related operations, including create, update and delete actions, as well as negative scenarios involving invalid data. Testing was performed using shell scripts that execute HTTP requests against the backend endpoints and validate the returned HTTP status codes and responses.

9.1 FUNCTIONAL TESTING

Functional testing was used to verify whether the implemented medication functionality **behaves according to the expected user and business requirements**. The main tested flow involved creating a medication with valid data, updating the created medication and then deleting it from the system. The test script first sends a valid medication payload containing fields such as name, category, stock, expiry date, batch number, price and supplier. If the backend returns a valid generated ID, the test considers the creation process successful. This confirms that the system can correctly process a valid medication creation request and persist the medication record in the backend.

The update scenario verifies whether an existing medication can be modified correctly. After a medication is created, the script sends a PUT request using the generated ID and checks whether the backend returns a successful HTTP status code. This validates that medication records can be updated after creation and that the API correctly handles modification operations. The delete scenario then sends a DELETE request for the same medication ID and verifies whether the system returns a success status, such as 204 No Content. This confirms that medication deletion works correctly for valid records.

9.2 API TESTING

API testing was performed directly against the backend endpoints using curl commands inside shell scripts. This approach allowed the team to validate backend behavior independently from the frontend interface. The tested endpoint was `/api/medication`, and the test covered the main CRUD operations through POST, PUT and DELETE requests. The script also validates responses using HTTP status codes and extracts generated IDs using jq, making the process repeatable and useful for regression testing.

In the initial development phase, the application included a manual medication creation endpoint used for early CRUD testing and basic validation scenarios. During this stage, a defect was identified where a negative stock value could be processed. The inventory model was later refactored into a transfer-oriented architecture, where medication records are no longer introduced manually into organizational inventories, but are added through validated transfer workflows between participating entities. The negative stock defect was fixed by implementing database-level integrity constraints through CHECK constraints on the `quantity_current` column of the `medication_batches` table, preventing stock values below zero. In addition, strict validation rules were introduced inside the SQL procedures responsible for accepting transfers, ensuring that inventory updates remain valid during transfer operations.

The API tests also included negative test cases, where invalid data was intentionally sent to the backend in order to verify error handling behavior. The tested invalid scenarios included sending a null medication name, sending an expiration date in an invalid format, sending text instead of a numeric stock value and sending a medication name that exceeds the database character limit. In these cases, the expected behavior was for the system to reject the request with a 4xx or 5xx status code. The terminal results show that these invalid requests were rejected correctly, confirming that part of the validation and global exception handling logic works as expected.

9.3 INTEGRATION TESTING

Integration testing focused on verifying the interaction between the API endpoint, backend validation logic and database persistence layer. The creation test confirms that a medication

object can be sent from an external client to the backend, processed by the application and stored with a generated identifier. The update and delete tests further validate that the backend can retrieve and modify existing database records based on their IDs. This confirms that the Controller, Service and Repository layers are working together correctly for the Medication API.

The tests also indirectly validate the global exception handling mechanism because invalid requests are processed and rejected instead of being silently accepted. This is important for integration reliability, since the frontend depends on predictable backend responses in order to display correct user feedback.

9.4 TEST SCENARIOS

The first scenario tested the creation of a medication using valid input data. The expected result was that the backend would accept the request and return a medication object with a valid ID. This test passed, confirming that valid medication creation works correctly.

The second group of scenarios tested invalid input data. A request with a null medication name was correctly rejected. A request with an invalid expiration date format was also rejected. Another test sent a string value instead of a numeric stock value, and the system rejected the request as expected. The final negative test sent a medication name longer than the allowed database limit, and the request was also rejected. These tests demonstrate that the backend performs input validation and prevents several invalid data formats from being stored.

A separate stock validation script tested whether the system prevents negative stock values. In one test execution, the backend accepted a medication with a negative stock value and returned a 201 status code, meaning that the validation failed for this case. This result is important because it revealed a business rule violation: stock quantities should never be allowed to become negative. The issue was identified through QA testing and should be treated as a defect requiring backend validation improvement.

9.5 VALIDATION RESULTS

The validation results show that the Medication API successfully passed the main functional tests for medication creation, update and deletion. The API also correctly rejected several invalid input cases, including null names, invalid date formats, invalid stock data types and overly long text values. These results confirm that the backend contains working validation and exception handling mechanisms for multiple scenarios.

However, the negative stock test revealed an important defect. The system accepted a stock value of -10 and returned a successful creation status. This behavior violates the business rule stating that stock cannot be negative. As a result, additional backend validation must be implemented to reject negative stock values before medication records are persisted. This finding demonstrates the value of QA testing, because it identified a real issue that could affect inventory accuracy and business logic correctness.

Overall, the testing phase confirmed that the main Medication API operations are functional, while also identifying an important validation gap related to negative stock values. The next QA step should be to retest the negative stock scenario after the backend validation logic is corrected.

```
~/Documents/Kron main*  
> ./teststock.sh  
  
[Test] Prevent negative stock...  
✗ FAILED: Sistemul a acceptat stoc negativ! (Status: 201)
```

```
},  
{  
  "id": 8,  
  "name": "Paracetamol",  
  "category": "Analgezic",  
  "stock": -10,  
  "expiryDate": "2026-12-31",  
  "batchNumber": null,  
  "price": null,  
  "supplier": null,  
  "status": null  
}
```

10 FUTURE IMPROVEMENTS

Although MedStock currently focuses on medication inventory management and transfer workflows between healthcare entities, the platform was designed with scalability and future expansion in mind. The modular architecture adopted during development allows new functionalities and user categories to be integrated without requiring major redesign efforts. Several future directions were identified during the planning phase in order to extend the platform beyond its initial scope and transform it into a broader healthcare ecosystem.

One of the most important future objectives involves expanding the system toward a **Business-to-Consumer (B2C) model**. In its current version, MediStock primarily targets hospitals, pharmacies and healthcare providers through internal inventory workflows. However, future iterations aim to broaden the platform audience by introducing direct interaction mechanisms between healthcare services and end users. This transition would allow the application to evolve beyond institutional inventory management and support a larger ecosystem involving patients and consumer-oriented healthcare services.

As part of this **B2C expansion**, future versions of the platform may introduce patient-oriented functionalities and include patients as active system participants. Instead of limiting interactions to managers, pharmacists and providers, patients could gain access to healthcare-related services and medication information through dedicated interfaces. Potential capabilities may include viewing prescribed medication information, receiving medication reminders, monitoring personal medication history or receiving notifications related to treatment schedules and medication availability. Integrating patients into the ecosystem would transform MedStock from an internal operational platform into a more comprehensive healthcare support system.

Another important future direction involves the development of a **mobile application intended** to improve accessibility and support faster interactions with the platform. Healthcare personnel frequently operate in dynamic environments where desktop access may not always be practical. A mobile implementation could provide users with immediate access to inventory information, transfer notifications and operational alerts directly from mobile devices. Pharmacists and managers could monitor inventory status and respond to critical events without depending exclusively on desktop systems.

A mobile platform could also become particularly valuable in the context of future patient integration. Patients could use mobile applications to receive healthcare notifications, track medication information and interact with personalized healthcare services. By supporting mobile environments, MediStock could improve accessibility and user engagement while expanding its overall usability.

Additional future improvements may include advanced AI capabilities, broader analytical dashboards, barcode and QR integration for medication identification, multi-hospital deployments

and interoperability with larger healthcare systems. The current architecture was intentionally designed to preserve flexibility and support gradual expansion without significantly increasing implementation complexity.

Overall, the long-term vision of MediStock extends beyond medication inventory tracking and aims toward creating a more connected, scalable and accessible healthcare ecosystem capable of supporting both institutional workflows and future patient-centered services.

11 CONCLUSIONS

The MediStock project represents the development of a medication inventory management platform designed to address several operational challenges commonly encountered in healthcare environments. Throughout the project, a complete software ecosystem was designed and implemented with the objective of improving inventory visibility, reducing medication waste and supporting more efficient decision-making processes. Rather than focusing solely on basic inventory operations, the platform integrates inventory workflows, transfer mechanisms and predictive functionalities into a centralized environment capable of supporting hospitals, pharmacies and healthcare providers.

The implemented solution combines multiple technological components into a unified architecture. A frontend application developed using Angular provides user interaction and visualization capabilities, while a Spring Boot backend manages authentication, business logic and inventory workflows through REST APIs and JWT-based stateless communication mechanisms. The persistence layer was implemented using Supabase, allowing medication information, users and transfer-related operations to be stored and managed efficiently. In addition to traditional inventory functionalities, the project also introduces an AI-assisted recommendation component capable of analyzing inventory conditions and generating transfer suggestions intended to reduce shortages and minimize expiration-related losses. The AI module represents one of the most important components of the system because it extends the platform beyond simple inventory visualization. By combining weighted historical demand forecasting, rule-based risk classification, Monte Carlo confidence estimation and explainable transfer recommendation logic, the system can identify both shortage risks and expiration-related waste opportunities. This allows MediStock to support proactive decision-making while preserving human control over final inventory actions.

During the development process, the team gained experience in several technical and organizational areas. From a technical perspective, the project provided practical exposure to layered software architectures, REST API development, authentication mechanisms, database design and integration workflows between frontend and backend components. The implementation of stateless authentication introduced important concepts regarding scalability and modern web application design. The AI module also provided valuable experience related to predictive logic, decision-support mechanisms and explainable analytical systems.

Beyond technical development, the project also introduced important lessons regarding software planning and collaboration processes. Activities related to Product Ownership, Business Analysis, backlog creation, requirement prioritization and release planning contributed toward understanding how technical implementation decisions are influenced by business objectives and stakeholder requirements. The project demonstrated the importance of defining product vision, identifying user needs and organizing development activities according to incremental release strategies.

Throughout the implementation process, several challenges were identified. Limited access to realistic healthcare datasets required the use of synthetic information for AI experimentation, while inventory validation and transfer logic required multiple iterations and testing activities. QA

processes and functional testing also revealed practical issues that required correction and refinement, reinforcing the importance of systematic validation procedures.

Although the current implementation focuses primarily on medication inventory workflows and transfer operations, the platform establishes a strong foundation for future expansion. The modular architecture allows the system to evolve toward larger deployment environments and additional healthcare functionalities. Future versions may introduce patient-centered capabilities, mobile applications, broader healthcare interactions and Business-to-Consumer functionalities capable of transforming MedStock into a more complete healthcare ecosystem.

Overall, MedStock demonstrates how modern software technologies, structured development methodologies and intelligent analytical mechanisms can be combined in order to create practical healthcare solutions. While currently positioned as a lightweight inventory-oriented platform, the project possesses significant potential for future growth and may evolve into a broader environment capable of connecting healthcare institutions, providers and patients through integrated and intelligent workflows.